

ACCESO A DATOS

(AD)

lridruejo

Tarea U5

2025 / 2026

ÍNDICE

FASE 1: Investigación y Entorno.	1
1. Justificación.	1
VENTAJAS	1
INCONVENIENTES	1
2. Configuración del proyecto.	2
FASE 2: Diseño de operaciones CRUD.	4
1. Conexión.	4
2. Create. Inserción de documentos.	5
3. Read. Consulta de documentos.	6
4. Update. Modificación de documentos.	7
5. Delete. Eliminación de documentos.	9

FASE 1: Investigación y Entorno.

1. Justificación.

Para la realización de esta tarea, he elegido **MongoDB Atlas**, la versión en la nube (Database as a Service) de MongoDB por su modelo de documentos flexible y escalable, ideal para aplicaciones como catálogos de e-commerce o de videojuegos donde los productos tienen atributos variables, sin necesidad de un esquema rígido, y por ser una solución gestionada en la nube que simplifica la administración y ofrece alto rendimiento.

Caso de uso real donde sería mejor esta opción que una BD relacional:

Una red social, donde los perfiles de usuario tienen campos variables no estructurados. MongoDB Atlas, al permitir el almacenamiento de datos heterogéneos (imágenes, textos, interacciones), sin necesidad de una estructura rígida, facilita el manejo de grandes volúmenes de información que cambia rápidamente. Además, su capacidad para escalar de manera horizontal lo convierte en una opción más eficiente frente a bases de datos relacionales en entornos de alta demanda.

VENTAJAS

- **Flexibilidad de esquema** → Al ser orientado a documentos, permite almacenar datos con atributos variables sin necesidad de modificar una estructura fija como en SQL. Por ejemplo, en videojuegos, cada personaje puede tener sus propios atributos sin necesidad de que todos compartan los mismos.
- **Escalabilidad horizontal automática** → MongoDB Atlas distribuye los datos entre múltiples servidores (sharding) de forma nativa, lo que facilita el crecimiento del catálogo sin perder rendimiento.

INCONVENIENTES

- **Coste elevado a gran escala** → Aunque tiene un nivel gratuito, al aumentar el volumen de datos y las operaciones, el precio puede dispararse en comparación con opciones autogestionadas.
- **Menor consistencia en operaciones complejas** → Al no soportar transacciones ACID de forma tan robusta como una base de datos relacional, puede ser menos adecuado para sistemas que requieran integridad transaccional estricta.

2. Configuración del proyecto.

Para poder conectarse a MongoDB Atlas desde un proyecto Java con Maven hay que añadir las siguientes líneas al archivo pom.xml, según la documentación oficial:

<https://www.mongodb.com/es/docs/drivers/java/sync/current/get-started/>

```
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>org.mongodb</groupId>
      <artifactId>mongodb-driver-bom</artifactId>
      <version>5.6.4</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>

<dependencies>
  <dependency>
    <groupId>org.mongodb</groupId>
    <artifactId>mongodb-driver-sync</artifactId>
  </dependency>
</dependencies>
```

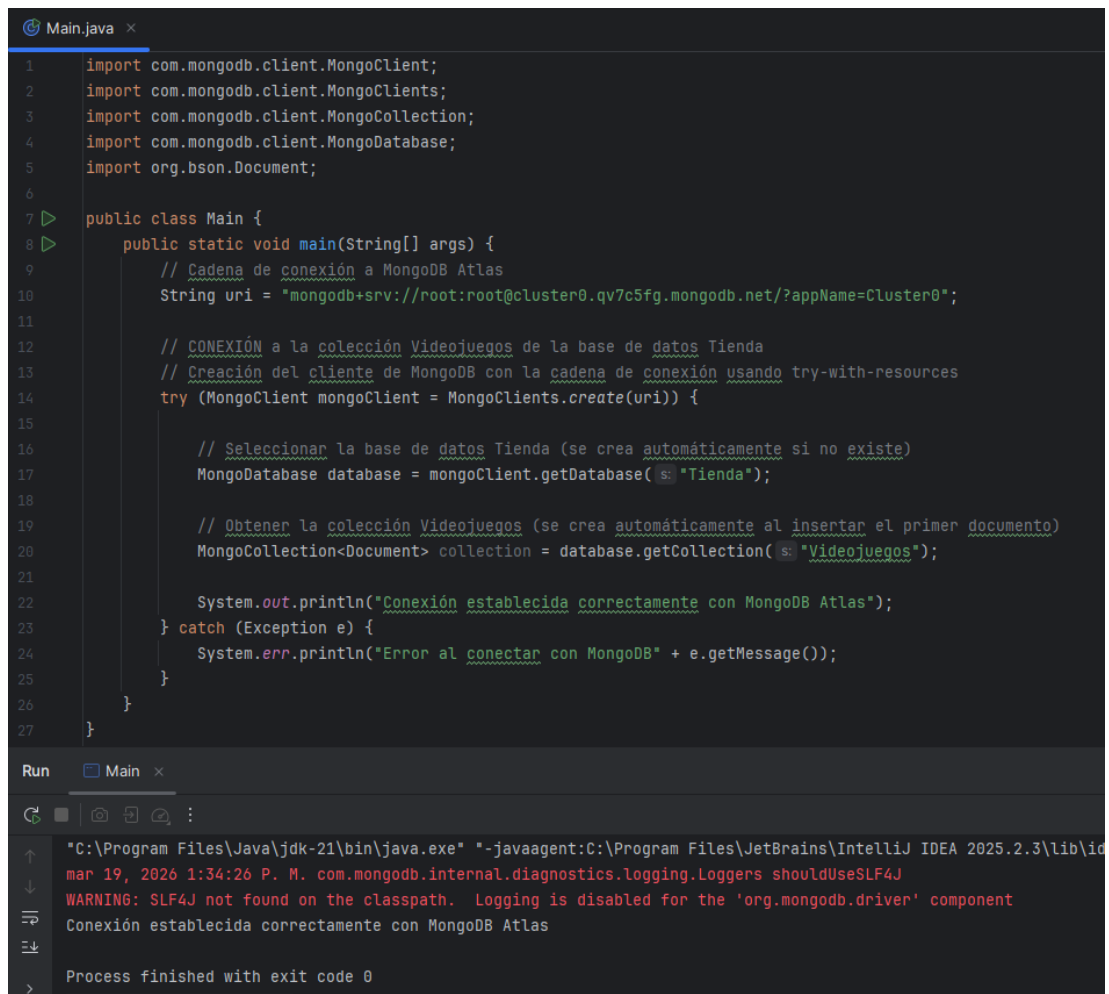
La primera parte, **mongodb-driver-bom**, lo que hace es añadir la Lista de materiales (BOM, Bill of Materials) que sirve para gestionar las versiones de las dependencias de MongoDB, entre ellas la siguiente: mongodb-driver-sync.

La dependencia **mongodb-driver-sync** es el controlador oficial sincrónico de MongoDB para Java, es decir, es la librería que permite que una aplicación Java se conecte y comunique con la base de datos MongoDB utilizando un modelo de programación sincrónico (bloqueante).

```
m pom.xml (Tarea5) x
1  <?xml version="1.0" encoding="UTF-8"?>
2  <project xmlns="http://maven.apache.org/POM/4.0.0"
3      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4      xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
5      <modelVersion>4.0.0</modelVersion>
6
7      <groupId>org.example</groupId>
8      <artifactId>Tarea5</artifactId>
9      <version>1.0-SNAPSHOT</version>
10
11     <properties>
12         <maven.compiler.source>21</maven.compiler.source>
13         <maven.compiler.target>21</maven.compiler.target>
14         <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
15     </properties>
16
17     <dependencyManagement>
18         <dependencies>
19             <dependency>
20                 <groupId>org.mongodb</groupId>
21                 <artifactId>mongodb-driver-bom</artifactId>
22                 <version>5.6.4</version>
23                 <type>pom</type>
24                 <scope>import</scope>
25             </dependency>
26         </dependencies>
27     </dependencyManagement>
28
29     <dependencies>
30         <dependency>
31             <groupId>org.mongodb</groupId>
32             <artifactId>mongodb-driver-sync</artifactId>
33         </dependency>
34     </dependencies>
35
36 </project>
```

FASE 2: Diseño de operaciones CRUD.

1. Conexión.



```
1 import com.mongodb.client.MongoClient;
2 import com.mongodb.client.MongoClients;
3 import com.mongodb.client.MongoCollection;
4 import com.mongodb.client.MongoDatabase;
5 import org.bson.Document;
6
7 public class Main {
8     public static void main(String[] args) {
9         // Cadena de conexión a MongoDB Atlas
10        String uri = "mongodb+srv://root:root@cluster0.qv7c5fg.mongodb.net/?appName=Cluster0";
11
12        // CONEXIÓN a la colección Videojuegos de la base de datos Tienda
13        // Creación del cliente de MongoDB con la cadena de conexión usando try-with-resources
14        try (MongoClient mongoClient = MongoClients.create(uri)) {
15
16            // Seleccionar la base de datos Tienda (se crea automáticamente si no existe)
17            MongoDatabase database = mongoClient.getDatabase("Tienda");
18
19            // Obtener la colección Videojuegos (se crea automáticamente al insertar el primer documento)
20            MongoCollection<Document> collection = database.getCollection("Videojuegos");
21
22            System.out.println("Conexión establecida correctamente con MongoDB Atlas");
23        } catch (Exception e) {
24            System.err.println("Error al conectar con MongoDB" + e.getMessage());
25        }
26    }
27 }
```

Run Main

```
"C:\Program Files\Java\jdk-21\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.2.3\lib\idea-ic
mar 19, 2026 1:34:26 P. M. com.mongodb.internal.diagnostics.Logging.Loggers shouldUseSLF4J
WARNING: SLF4J not found on the classpath. Logging is disabled for the 'org.mongodb.driver' component
Conexión establecida correctamente con MongoDB Atlas

Process finished with exit code 0
```

Para las operaciones CRUD, he usado este tutorial como guía:

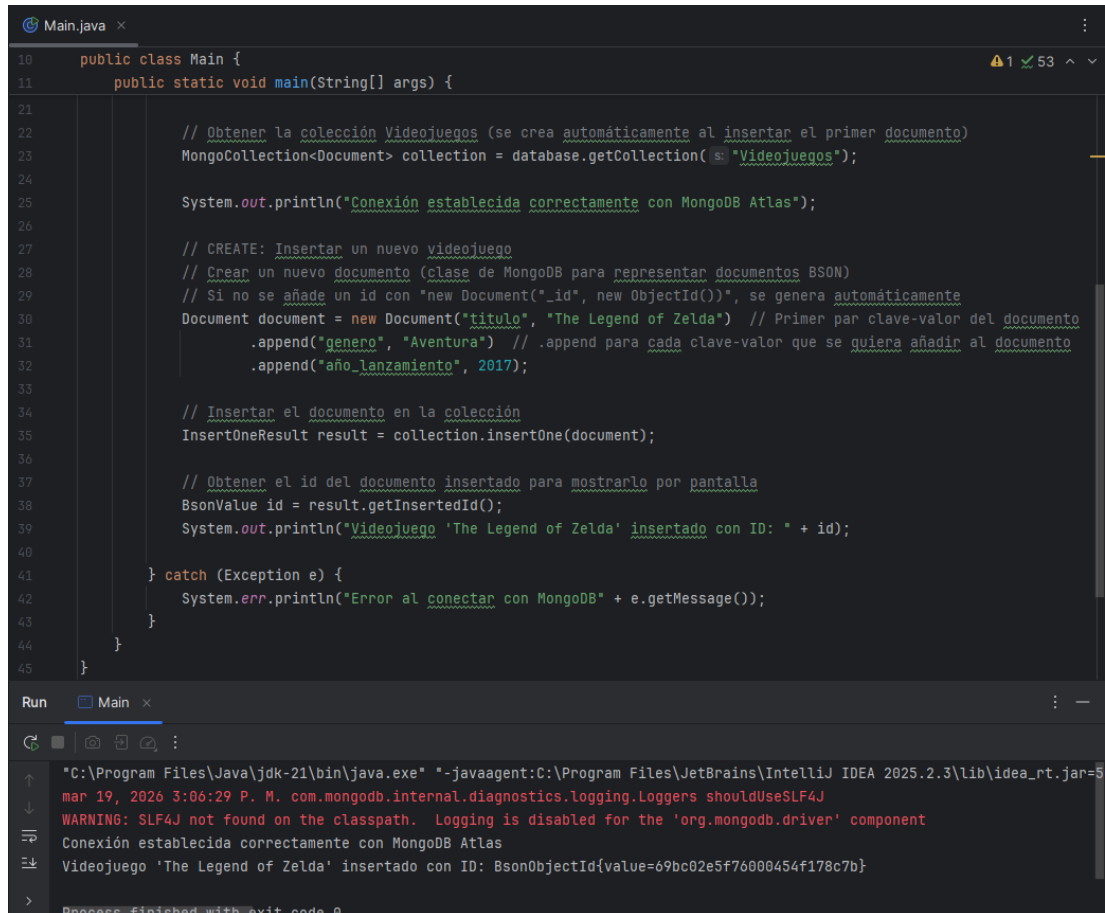
<https://learn.mongodb.com/learn/course/mongodb-crud-operations-in-java>

Para las clases **Filters** y **Updates** he usado esta documentación como guía:

<https://www.mongodb.com/docs/drivers/java/sync/current/builders/>

2. Create. Inserción de documentos.

Teniendo una conexión activa y una colección de una base de datos seleccionada [ver [Conexión](#)], se insertan nuevos documentos con el método de la colección **“insertOne(documento)”** para un solo documento o **“insertMany(array de documentos)”** para múltiples documentos.



```
10 public class Main {
11     public static void main(String[] args) {
12
13         // Obtener la colección Videojuegos (se crea automáticamente al insertar el primer documento)
14         MongoClient database = new MongoClient("mongodb://localhost:27020");
15         MongoCollection<Document> collection = database.getCollection("Videojuegos");
16
17         System.out.println("Conexión establecida correctamente con MongoDB Atlas");
18
19         // CREATE: Insertar un nuevo videojuego
20         // Crear un nuevo documento (clase de MongoDB para representar documentos BSON)
21         // Si no se añade un id con "new Document("_id", new ObjectId())", se genera automáticamente
22         Document document = new Document("titulo", "The Legend of Zelda") // Primer par clave-valor del documento
23             .append("genero", "Aventura") // .append para cada clave-valor que se quiera añadir al documento
24             .append("año_lanzamiento", 2017);
25
26         // Insertar el documento en la colección
27         InsertOneResult result = collection.insertOne(document);
28
29         // Obtener el id del documento insertado para mostrarlo por pantalla
30         BsonValue id = result.getInsertedId();
31         System.out.println("Videojuego 'The Legend of Zelda' insertado con ID: " + id);
32
33     } catch (Exception e) {
34         System.err.println("Error al conectar con MongoDB" + e.getMessage());
35     }
36 }
37 }
```

Run Main x

```
"C:\Program Files\Java\jdk-21\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.2.3\lib\idea_rt.jar=5
mar 19, 2026 3:06:29 P. M. com.mongodb.internal.diagnostics.logging.Loggers shouldUseSLF4J
WARNING: SLF4J not found on the classpath. Logging is disabled for the 'org.mongodb.driver' component
Conexión establecida correctamente con MongoDB Atlas
Videojuego 'The Legend of Zelda' insertado con ID: BsonObjectId{value=69bc02e5f76000454f178c7b}
Process finished with exit code 0
```

3. Read. Consulta de documentos.

Teniendo una conexión activa y una colección de una base de datos seleccionada [ver [Conexión](#)], se obtiene un documento con el método de la colección “**find()**” al que se le pasa como argumento un método estático de la clase **Filters**. Se usa “**first()**” para obtener solamente el primer documento que se encuentre (aunque al buscar por id no haría falta porque el valor de `_id` es único).

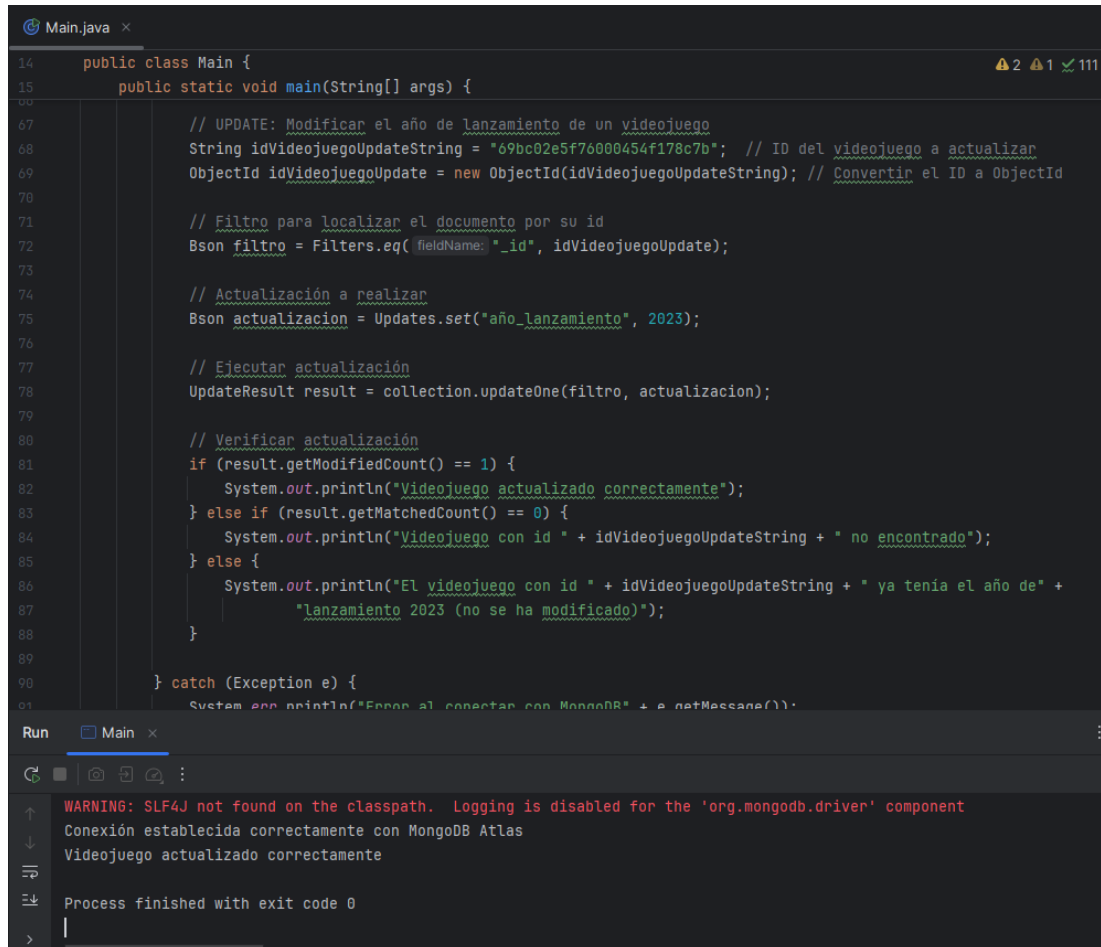
```
Main.java x
11 public class Main {
12     public static void main(String[] args) {
13
14         // READ: Consultar la base de datos y obtener el videojuego utilizando su id
15         String idString = "69bc02e5f76000454f178c7b"; // ID del videojuego a obtener
16         ObjectId idVideojuego = new ObjectId(idString); // Convertir el id a ObjectId
17
18         // Buscar el documento que tenga como _id el id del videojuego a obtener
19         Document videojuego = collection.find(Filters.eq("field_name: "_id", idVideojuego)).first();
20
21         // Verificar si se encontró el videojuego (documento)
22         if (videojuego != null) {
23             // Mostrar los datos
24             System.out.println("VIDEOJUEGO:");
25             System.out.println("ID: " + videojuego.get("_id"));
26             System.out.println("Título: " + videojuego.get("titulo"));
27             System.out.println("Género: " + videojuego.get("genero"));
28             System.out.println("Año de lanzamiento: " + videojuego.get("año_lanzamiento"));
29         } else {
30             System.out.println("Videojuego con id " + idString + " no encontrado");
31         }
32
33     } catch (Exception e) {
34         System.err.println("Error al conectar con MongoDB" + e.getMessage());
35     }
36 }
37 }
```

Run Main x

Conexión establecida correctamente con MongoDB Atlas
VIDEOJUEGO:
ID: 69bc02e5f76000454f178c7b
Título: The Legend of Zelda
Género: Aventura
Año de lanzamiento: 2017

4. Update. Modificación de documentos.

Teniendo una conexión activa y una colección de una base de datos seleccionada [ver [Conexión](#)], se modifica un documento con el método de la colección **“updateOne()”** (para modificar más de un documento al mismo tiempo se usaría **“updateMany()”**) al que se le pasan como argumentos un filtro (clase **Filters** igual que en el Read [ver [Read. Consulta de documentos](#)]) y la actualización a realizar (método estático **“set()”** de la clase **Updates** para una sola modificación, **“combine()”** para combinar varias modificaciones).

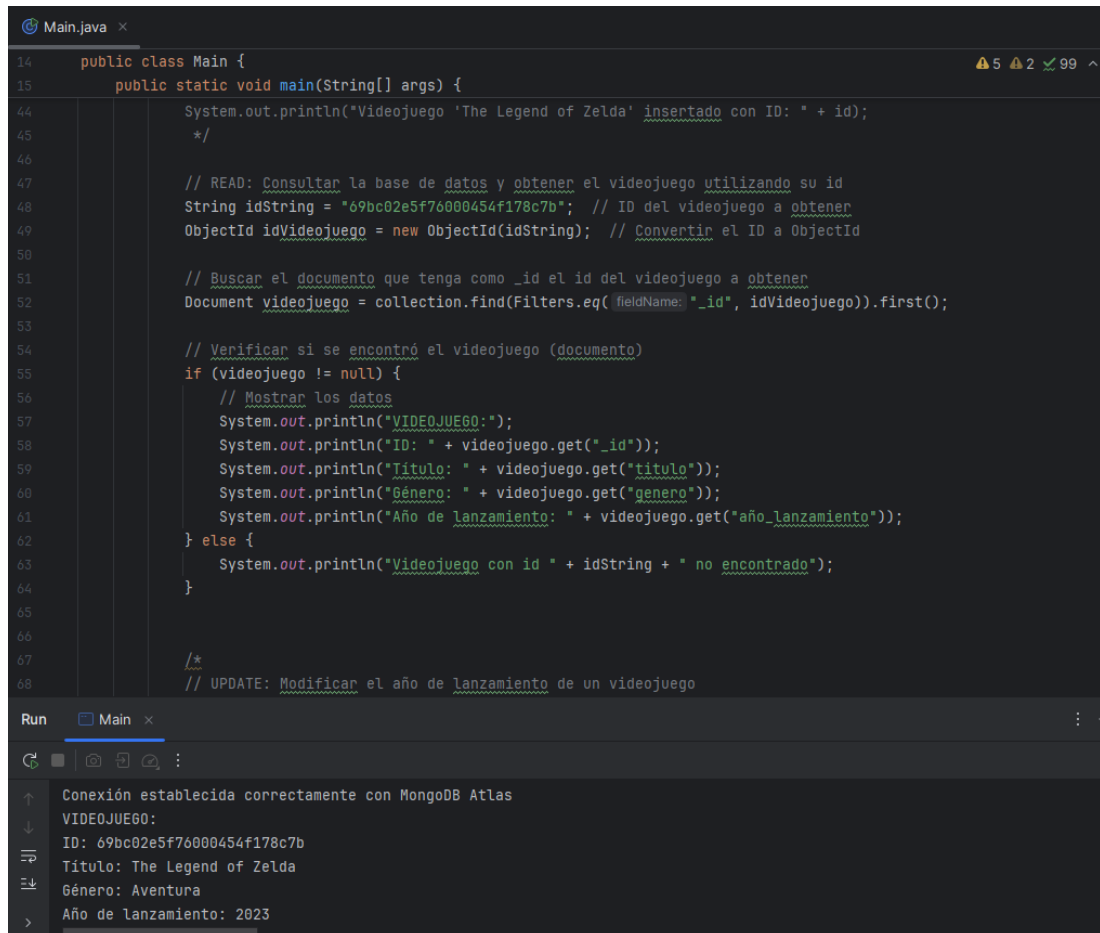


```
14 public class Main {
15     public static void main(String[] args) {
16
17         // UPDATE: Modificar el año de lanzamiento de un videojuego
18         String idVideojuegoUpdateString = "69bc02e5f76000454f178c7b"; // ID del videojuego a actualizar
19         ObjectId idVideojuegoUpdate = new ObjectId(idVideojuegoUpdateString); // Convertir el ID a ObjectId
20
21         // Filtro para localizar el documento por su id
22         Bson filtro = Filters.eq("id", idVideojuegoUpdate);
23
24         // Actualización a realizar
25         Bson actualizacion = Updates.set("año_lanzamiento", 2023);
26
27         // Ejecutar actualización
28         UpdateResult result = collection.updateOne(filtro, actualizacion);
29
30         // Verificar actualización
31         if (result.getModifiedCount() == 1) {
32             System.out.println("Videojuego actualizado correctamente");
33         } else if (result.getMatchedCount() == 0) {
34             System.out.println("Videojuego con id " + idVideojuegoUpdateString + " no encontrado");
35         } else {
36             System.out.println("El videojuego con id " + idVideojuegoUpdateString + " ya tenía el año de " +
37                 "lanzamiento 2023 (no se ha modificado)");
38         }
39
40     } catch (Exception e) {
41         System.err.println("Error al conectar con MongoDB: " + e.getMessage());
42     }
43 }
```

Run Main x

WARNING: SLF4J not found on the classpath. Logging is disabled for the 'org.mongodb.driver' component
Conexión establecida correctamente con MongoDB Atlas
Videojuego actualizado correctamente
Process finished with exit code 0

Si volvemos a ejecutar la consulta [ver [Read](#). Consulta de documentos.], comprobamos que se ha modificado el año de lanzamiento de ese videojuego.



The screenshot shows an IDE with a Java file named 'Main.java'. The code is a public class 'Main' with a static void 'main' method. It prints a message, then uses a MongoDB ObjectId to find a document in a collection. The document is printed with its fields: ID, Title, Genre, and Release Year. The output shows the release year as 2023.

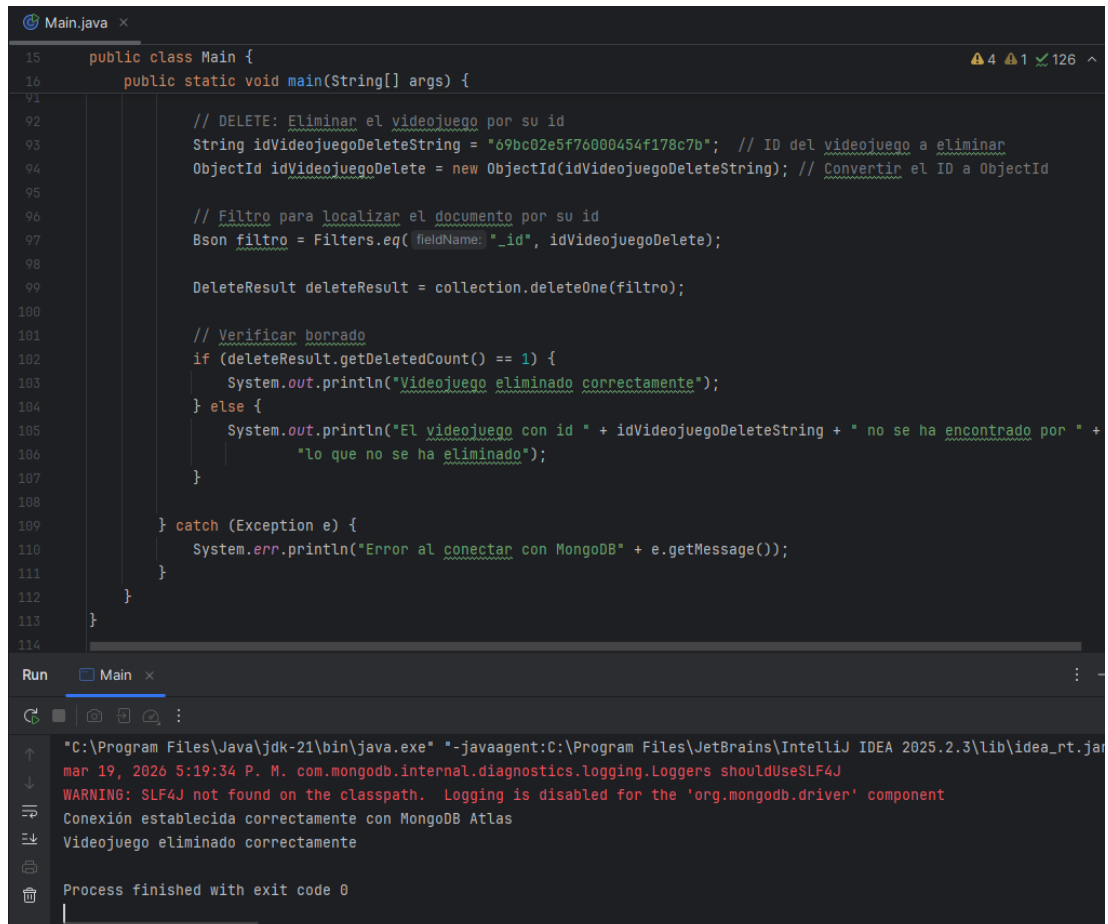
```
14 public class Main {
15     public static void main(String[] args) {
44         System.out.println("Videojuego 'The Legend of Zelda' insertado con ID: " + id);
45         /*
46
47         // READ: Consultar la base de datos y obtener el videojuego utilizando su id
48         String idString = "69bc02e5f76000454f178c7b"; // ID del videojuego a obtener
49         ObjectId idVideojuego = new ObjectId(idString); // Convertir el ID a ObjectId
50
51         // Buscar el documento que tenga como _id el id del videojuego a obtener
52         Document videojuego = collection.find(Filters.eq( fieldName: "_id", idVideojuego)).first();
53
54         // Verificar si se encontró el videojuego (documento)
55         if (videojuego != null) {
56             // Mostrar los datos
57             System.out.println("VIDEOJUEGO:");
58             System.out.println("ID: " + videojuego.get("_id"));
59             System.out.println("Titulo: " + videojuego.get("titulo"));
60             System.out.println("Género: " + videojuego.get("genero"));
61             System.out.println("Año de lanzamiento: " + videojuego.get("año_lanzamiento"));
62         } else {
63             System.out.println("Videojuego con id " + idString + " no encontrado");
64         }
65
66
67         /*
68         // UPDATE: Modificar el año de lanzamiento de un videojuego
```

Run Main x

Conexión establecida correctamente con MongoDB Atlas
VIDEOJUEGO:
ID: 69bc02e5f76000454f178c7b
Titulo: The Legend of Zelda
Género: Aventura
Año de lanzamiento: 2023

5. Delete. Eliminación de documentos.

Teniendo una conexión activa y una colección de una base de datos seleccionada [ver [Conexión](#).], se elimina un documento con el método de la colección **“.deleteOne()”** (para eliminar más de un documento al mismo tiempo se usaría **“.deleteMany()”**) al que se le pasa como argumento un filtro (clase **Filters** igual que en el Read [ver [Read](#). Consulta de documentos.]).

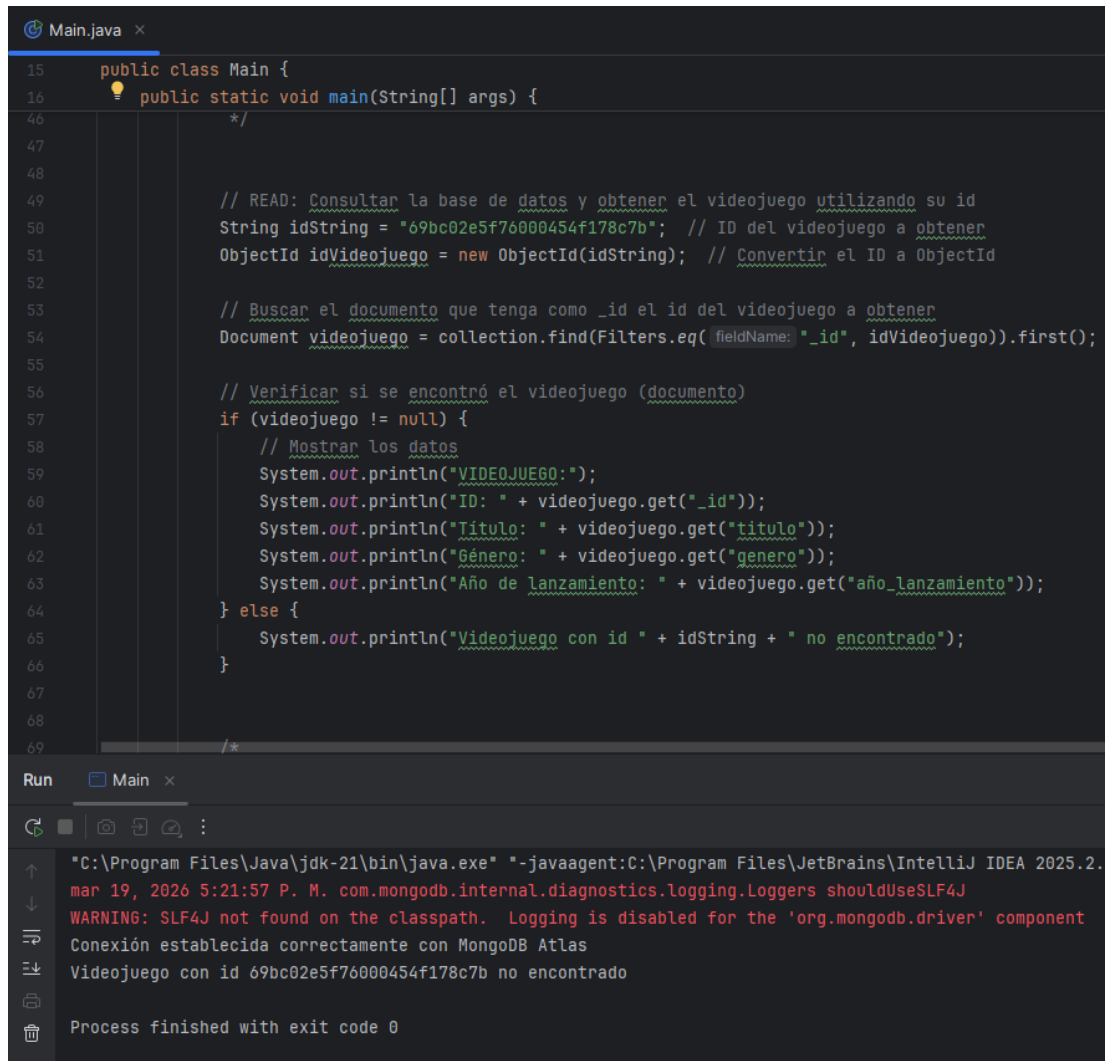


```
15 public class Main {
16     public static void main(String[] args) {
17
18         // DELETE: Eliminar el videojuego por su id
19         String idVideojuegoDeleteString = "69bc02e5f76000454f178c7b"; // ID del videojuego a eliminar
20         ObjectId idVideojuegoDelete = new ObjectId(idVideojuegoDeleteString); // Convertir el ID a ObjectId
21
22         // Filtro para localizar el documento por su id
23         Bson filtro = Filters.eq( "id", idVideojuegoDelete);
24
25         DeleteResult deleteResult = collection.deleteOne(filtro);
26
27         // Verificar borrado
28         if (deleteResult.getDeletedCount() == 1) {
29             System.out.println("Videojuego eliminado correctamente");
30         } else {
31             System.out.println("El videojuego con id " + idVideojuegoDeleteString + " no se ha encontrado por " +
32                 "lo que no se ha eliminado");
33         }
34
35     } catch (Exception e) {
36         System.err.println("Error al conectar con MongoDB" + e.getMessage());
37     }
38 }
39 }
```

Run Main

"C:\Program Files\Java\jdk-21\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.2.3\lib\idea_rt.jar" mar 19, 2026 5:19:34 P. M. com.mongodb.internal.diagnostics.logging.Loggers shouldUseSLF4J
WARNING: SLF4J not found on the classpath. Logging is disabled for the 'org.mongodb.driver' component
Conexión establecida correctamente con MongoDB Atlas
Videojuego eliminado correctamente
Process finished with exit code 0

Si volvemos a ejecutar la consulta [ver [Read](#). Consulta de documentos.], comprobamos que ese videojuego ya no existe.



The screenshot displays an IDE with a Java file named 'Main.java' and a 'Run' console window. The code in 'Main.java' is as follows:

```
15 public class Main {
16     public static void main(String[] args) {
46         /*
47
48
49         // READ: Consultar la base de datos y obtener el videojuego utilizando su id
50         String idString = "69bc02e5f76000454f178c7b"; // ID del videojuego a obtener
51         ObjectId idVideojuego = new ObjectId(idString); // Convertir el ID a ObjectId
52
53         // Buscar el documento que tenga como _id el id del videojuego a obtener
54         Document videojuego = collection.find(Filters.eq( fieldName: "_id", idVideojuego)).first();
55
56         // Verificar si se encontró el videojuego (documento)
57         if (videojuego != null) {
58             // Mostrar los datos
59             System.out.println("VIDEOJUEGO:");
60             System.out.println("ID: " + videojuego.get("_id"));
61             System.out.println("Título: " + videojuego.get("titulo"));
62             System.out.println("Género: " + videojuego.get("genero"));
63             System.out.println("Año de lanzamiento: " + videojuego.get("año_lanzamiento"));
64         } else {
65             System.out.println("Videojuego con id " + idString + " no encontrado");
66         }
67
68
69         /*
```

The 'Run' console window shows the following output:

```
"C:\Program Files\Java\jdk-21\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.2.
mar 19, 2026 5:21:57 P. M. com.mongodb.internal.diagnostics.logging.Loggers shouldUseSLF4J
WARNING: SLF4J not found on the classpath. Logging is disabled for the 'org.mongodb.driver' component
Conexión establecida correctamente con MongoDB Atlas
Videojuego con id 69bc02e5f76000454f178c7b no encontrado

Process finished with exit code 0
```